Querying a Codebase with NVIDIA Auto Ontology

Will Cook

5 November 2026

A source-to-SQL adapter, demonstrated on Auto Ontology's own repository

Abstract

Which code semantics can an outside project supply to NVIDIA Auto Ontology through its existing data and model contracts, and do they change the agent's answers? We built **code-ontology-bridge**, a Python extractor that turns source files, symbols, calls and existing docstrings into a code database, with table descriptions (a semantic model in Auto Ontology's import format). An optional semantics layer adds six ready-made SQL views for calls, callbacks, reachability, file and symbol facts, dependencies and inheritance, four of them compared with a separate Python computation on randomly generated codebases. We tested both on 48 questions about four repositories, written by separate AI authors from the source and the bare code database, without seeing the table descriptions or the views, re-derived by independent AI annotators and frozen before any model ran. An answer counts as robust only if its SQL also returns the right rows on eight counterfactual versions of the database. After the runs, an audit corrected 21 reference queries that missed a restriction stated in their question, and every saved response was rescored. Inside Auto Ontology, with only the code database connected, the agent with DeepSeek v4.1 Flash answered 10.3 of the 12 Auto Ontology questions robustly (mean of three runs), against 3.2 for the same model asked those questions directly; importing the table descriptions did not raise that score. Asked directly, the table descriptions raised DeepSeek from 18.0 to 27.2 robust answers of 48 and NVIDIA Nemotron 3.5 Lightning from 2.4 to 5.4 (mean of five runs); adding the views took Nemotron to 7.6. The corrections raised the first figure from 7.3 and lowered the second from 34.0. All questions, answer keys, saved responses and both score sets are published.

1. The question and the contribution

Suppose a developer changes `resolve_semantic_fks`, the function that resolves semantic foreign keys in Auto Ontology. Which callers reach it? Searching for its name finds an immediate caller. Reaching the scheduler also requires following a wrapper and recognising a function passed as an argument. The question spans several files and two kinds of relationship: direct calls and callback references.

A code database extracted from the source can record those relationships once and make them available to SQL. NVIDIA Auto Ontology already has the other half of this workflow: an agent that retrieves semantic context, writes SQL and executes it against a connected database. This project connects the two. Instead of asking Auto Ontology about a sales database, we ask it about a database of its own implementation.

The project delivers four things:

- **A source adapter.** A standalone extractor records Python source facts and exports DuckDB rows, model YAML and an evaluation dataset using Auto Ontology's existing contracts.
- **An exercised integration.** A compatibility patch configures public model clients and exposes an existing SQL routing option at the chat boundary. The retained run connects ingestion, model import, semantic compilation and question answering.
- **A code semantics layer** (called the ready-made queries in the README and on the website).

Six SQL views, four of them compared with a separate Python computation, that define direct calls, callback references, reachability, file and symbol facts, dependencies and inheritance once, documented in the native model.

- **A blind, repeated study.** 48 questions about four repositories, written without sight of the table descriptions or the semantics layer, scored on real and counterfactual databases, with every run, response and failure published.

The source-map approach comes from **CodeMap**, Will Cook's earlier source-navigation tool. This bridge adapts that approach into an installable package; it does not depend on CodeMap's private interface. NVIDIA supplies the database agent and its semantic catalog. The bridge supplies the code database and its table descriptions.

2. How the source reaches the agent

Auto Ontology needs both a live database to query and semantic context that explains the database. The bridge exports them separately. Figure 1 follows the two paths to SQL execution.

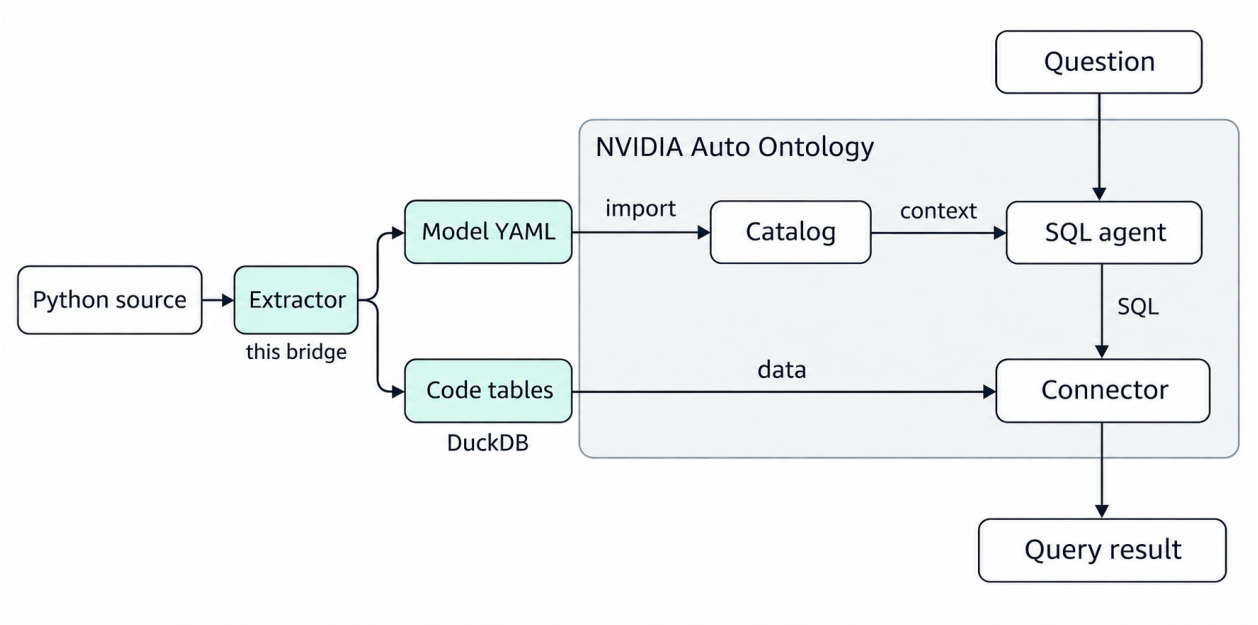


Figure 1: Two exports serve different purposes. DuckDB supplies source-code rows; model YAML supplies descriptions and relationships through import and compilation. The outlined components belong to Auto Ontology. The extractor is this bridge; the catalog is the semantic catalog and the connector is the existing DuckDB connector. This simplified query path omits ingestion services, model providers and answer narration.

The **DuckDB export** contains eight tables: files, docstring headings, symbols, class bases, calls, imports, test links and a source snapshot. Two views summarise incoming calls and public symbols without test associations. Auto Ontology reads these through its existing DuckDB connector.

The **model YAML** describes tables, terms, attributes and joins. It declares keys that the pinned DuckDB connector does not report. The live runner first ingests the database, maps the exported catalog identifiers to the server's identifiers, imports the model and compiles its semantic catalog. Importing YAML alone does not create a queryable database.

A third export supplies DDL, CSV data and reference questions in NVIDIA's evaluation format. It provides a route toward a controlled comparison, but the full upstream benchmark harness was not run for this study.

What is a row, and what is a definition?

The extractor uses Python's AST and preserves source paths and line ranges. A function becomes a row in `code.symbols`. Its docstring becomes a value in `summary`. A call becomes a row in `code.calls`, with identifiers for its caller and, when resolved, its callee. Joining symbols to files lets a query recover locations or exclude test files.

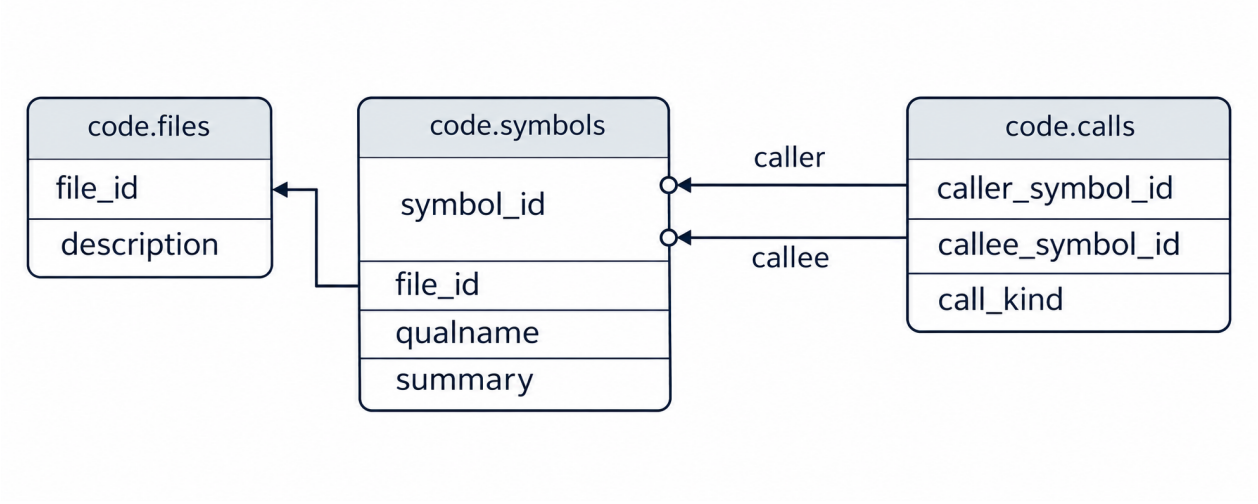


Figure 2: Selected columns show the joins used by a caller query. Each call references its caller and, when resolved, its callee; each symbol references its file. Descriptions of particular files and functions remain data values. The semantic model describes what the tables and columns mean.

This distinction keeps the model small: it defines a symbol as an entity type, rather than creating a separate semantic term for every function. Source descriptions are copied from existing docstrings. Missing descriptions stay empty; the extractor does not ask a model to invent them. The interface specification gives all columns, identity rules and resolution categories.

One source occurrence, two representations

In `semantic_scheduler.py` under `auto_ontology/ingestion_service/`, line 203 of the pinned source passes the wrapper as an argument:

```
tables_processed = await asyncio.to_thread(
    run_semantic_compilation, database_name
)
```

The extracted row includes these fields:

```
call_kind: callback    resolution: imported
confidence: low       line: 203
```

The caller is `SemanticScheduler._run_once`, whose definition starts at line 132 in the scheduler file. The callee is `run_semantic_compilation`, defined in `compile.py` under `auto_ontology/semantic/`.

These selected fields from the frozen native model input show how Code symbol maps to the data:

```
id: cob:auto_ontology_code:term:symbols
name: Code symbol
represents:
  - cob:auto_ontology_code:table:code.symbols
columns_attributes:
  - id: cob:auto_ontology_code:attribute:code.symbols.symbol_id
    name: symbol id
    column_id: cob:auto_ontology_code:column:code.symbols.symbol_id
```

Below, IDs omit the shared `cob:auto_ontology_code:` prefix. The Call site term represents `table:code.calls`. Its semantic foreign keys connect two columns to the symbol-ID attribute:

- `column:code.calls.caller_symbol_id`
- `column:code.calls.callee_symbol_id`

Both target `attribute:code.symbols.symbol_id`. These definitions describe the joins; the particular functions and call occurrences remain database rows. The `summary` attribute's authored description explains a source-docstring column, while each row's `summary` contains the original docstring. Both the queryable rows and the imported semantic context are needed.

The runner remaps exported catalog IDs before import. The frozen exported input and a later `/api/terms` response are retained; the submitted YAML, ID mapping and full compiled-model export are not. The terms response is a partial catalog readback and does not show which definitions a question retrieved. The inspection backend exposes these stages separately and supplies hash-checked source excerpts at build time.

What changed upstream?

The data connector, model import and SQL pipeline already existed. The source extractor and exports need no changes to NVIDIA's code or model credentials. The recorded live run used Auto Ontology at `cdaf61e1bd0cf3820d11d72b43bb460816aebbd3`. A separate twelve-file compatibility patch configures public model clients, forwards the existing `calculation_only` option through the chat API and extends a placeholder validator. It adds no code-specific connector, SQL engine or query planner.

Fixed SQL routing matters here because a question such as "What does this module do?" can sound like a metadata question. In this dataset, the answer is a value in a table. The patch selects the row-querying path for every question in the run. Its documentation and validation record 611 passing upstream tests in the query-pipeline, chat and utility scopes, passing lint, and a clean `git apply` to a fresh pinned checkout.

3. Following the caller chain

Question 11 asks which resolved caller symbols reach `resolve_semantic_fks`, directly or indirectly, excluding tests and reporting the shortest depth. The full request also prescribed column names and types and instructed the agent to traverse resolved calls backwards, recursively, including callback references. Given those instructions, Auto Ontology generated recursive SQL over `code.calls` and returned four callers.

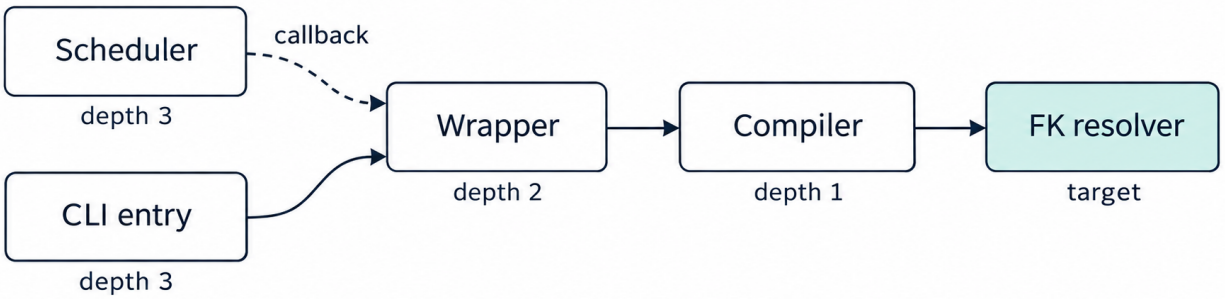


Figure 3: Four callers reach the foreign-key resolver through a wrapper and compiler. The table below identifies each source symbol. Arrows point from caller to callee, and SQL follows them in reverse. Solid edges are direct calls. The dashed edge records a callback passed to asyncio.to_thread, not proof that it ran.

Diagram label	Source symbol	Depth
Compiler	`_run_semantic_compilation`	1
Wrapper	`run_semantic_compilation`	2
Scheduler	`SemanticScheduler._run_once`	3
CLI entry	`main`	3

The target labelled “FK resolver” is `resolve_semantic_fks`; it is not a returned caller.

The locally computed inspection presets return three callers with direct calls alone and four when callback references are included; only the scheduler is added. This is a traversal of saved rows, not another model answer.

The scheduler passes `run_semantic_compilation` to `asyncio.to_thread`. The bridge records that argument as a callback reference, which connects the scheduler to the same chain as the ordinary caller `main`. The generated query follows reverse edges, avoids cycles and groups by full caller identity to select the minimum depth.

The saved response includes the request, intermediate events, final SQL and complete symbol identifiers. The request contains a hint about reverse traversal and callbacks; it is not an unaided discovery test. No saved SQL example was imported for this run. The answer establishes reachability within the extracted map. Dynamic calls can be missing, and a callback reference does not establish execution.

4. Evaluation: questions the system never saw

4.1 Design

The study (protocol) asks how much each part of what the bridge exports contributes to correct answers, on questions written without knowledge of it. The order of work is fixed by public commits.

1. **The semantics layer came first.** Six DuckDB views over the exported tables define common code-question concepts once: `call_edges` (direct calls and callback references for each caller and callee), `reachability` (the shortest chain of calls between two symbols, through direct calls only and through callbacks too, unbounded and safe on cycles), `symbol_profile`, `file_profile` (including each file's folder), `file_dependencies` and `class_ancestry`. Every view and column is documented in the native model with keys and joins. Property tests compare four of the views (`call_edges`, `reachability`, `class_ancestry` and the caller count in `symbol_profile`) with an independent Python computation on 60 random code maps. The views and their tests were committed before any question existed; their descriptions were extended after the study.
2. **Four authors wrote 12 questions each**, one per repository: NVIDIA Auto Ontology at a current upstream commit (474 Python files), pallets/click (91), encode/httpx (60) and Textualize/rich (213). The authors were separate AI agents with the source checkout and a database containing only the base tables and two base views, instructed not to open the layer, its tests or the study folder. They chose topics freely, wrote reference SQL and checked each answer against the source. Section 4.3 describes the references corrected after the runs.
3. **A second annotator per repository** received only the question text and output columns and wrote its own SQL. All 48 answers agreed with the authors'. Wherever the annotator named a second plausible reading, the question was reworded so one reading remains: 17 questions, with no reference changed and none removed.
4. **The questions, references and protocol were committed** (`37b4f05`) before any model saw them.

4.2 Arms and scoring

Four briefings of the same model, querying the same database:

- **A, schema.** Table and column names and types.
- **B, table descriptions.** A plus every table's and column's description, the keys and the joins.
- **V, views.** A plus the semantics layer's views listed by name and type, without descriptions; a control for the views alone.
- **S, semantics layer.** B plus the layer's views, documented columns, keys, joins and a short glossary of definitions.

Inside Auto Ontology, the agent with its retrieval, validation and repair, the source facts and the chat model are fixed. Every configuration starts from the same code database. Three configurations differ: **nothing** imported, so the agent works from the catalog Auto Ontology builds from the database by itself, the **table descriptions** imported, or the **semantics layer**, which adds the six views to the database and imports the model that describes them.

DeepSeek v4.1 Flash and NVIDIA Nemotron 3.5 Lightning answered through OpenRouter. Arms A, B, V and S set temperature 0, reasoning off and at most 4,000 output tokens in every request and ran five complete times. The Auto Ontology arms ran three times each on the 12 Auto Ontology questions through Auto Ontology's own chat clients, with its own output limits and the provider's default temperature and reasoning settings.

Each answer is scored twice. **Exact rows**: the returned rows equal the reference rows, with named and typed columns and the requested order. **Robust**: the same SQL also returns the reference SQL's rows on eight counterfactual copies of the repository's database, which add calls, callbacks, same-named functions, methods, imports and bases, remove some calls and change docstrings and sizes. A query can return the right rows on one database by coincidence; following Zhong, Yu and Klein (2020), the counterfactual databases expose many such queries, so the robust score approximates semantic accuracy. The reference answer changes on at least one counterfactual database for 44 of

the 48 questions. Substituting the reference SQL for the model scores 48 of 48 on both measures.

4.3 Results

Reference corrections. After the runs, an outside reviewer found that the reference query for `ao-03` ignores its question's restriction to functions and methods. The extracted symbols also include classes. A class appears as a caller when its body makes a call, and as a callee when it is constructed.

Four auditors then compared all 48 reference queries with their questions' wording, without seeing any model answer or score. They found 21 reference queries that missed a condition stated in the question; 15 of these lacked the same function-or-method restriction. Five questions can be read two ways, and the scorer now accepts either reading. The corrected queries were committed (`9097b80`) before rescoring, and the saved answers were rescored without new model calls. The original scores remain published.

No corrected query changes its answer on a real database, so exact-row scores are unchanged. Robust scores do change. Directly prompted models usually left out the restriction, as the original references did, so they lose robust answers. Auto Ontology's agent with DeepSeek usually wrote the restriction, so it gains. The tables below use the corrected references.

A model asked directly, robust answers out of 48, mean of five runs (lowest and highest in brackets):

Briefing	DeepSeek v4.1 Flash	Nemotron 3.5 Lightning
A, schema	18.0 (15–20)	2.4 (2–4)
B, table descriptions	27.2 (24–30)	5.4 (4–6)
V, views without descriptions	25.4 (23–29)	2.8 (2–5)
S, semantics layer	26.8 (25–29)	7.6 (5–10)

Inside Auto Ontology, robust answers out of 12, mean of three runs:

Auto Ontology is given	DeepSeek v4.1 Flash	Nemotron 3.5 Lightning
Code database, nothing imported	10.3 (10–11)	0.7 (0–1)
+ table descriptions	8.7 (8–9)	2.3 (0–5)
+ table descriptions and semantics layer	10.7 (10–12)	2.3 (1–4)

The table descriptions make a large difference to a model asked directly. With DeepSeek, they add about nine robust answers out of 48; question by question, nine improve and one regresses. With Nemotron the semantics layer lifts robust answers from 2.4 to 7.6; six questions improve and none regress.

The layer's value depends on the model and the agent. For DeepSeek asked directly, the table descriptions and the semantics layer tie (five questions improve and six regress). For Nemotron the layer is the best briefing, and adding the layer's definitions to its views improves six questions with none regressing. For DeepSeek the views alone already help, and their definitions add little on these questions.

Inside Auto Ontology, the code database alone answers 9 of the 12 questions in every run. These include three of the six questions their author rated hard: agents built in the product but never in its tests, functions that open a transaction without using the connection, and the five most widely used data-access functions; DeepSeek asked directly with table names answered none of those three in five runs. With nothing imported the agent answers 10.3 of 12 robustly (runs of 10, 10 and 11) and 11.7 on the real database; the semantics layer gives 10.7 (10, 12 and 10) and the

table descriptions alone 8.7 (8, 9 and 9). With Nemotron both imports raise the score from 0.7 to 2.3. The agent with DeepSeek usually applied the questions' restrictions on kind of symbol, which directly prompted models usually missed.

4.4 Limits

The questions were written by AI authors under the protocol's rules, not by the repositories' maintainers, and their topics reflect what the base database can answer. Reference answers are correct for the extracted database. The authors' source checks found facts the extractor misses: calls through a local alias, calls to functions with `@overload` stubs, names reached through package re-exports and calls on objects of unknown type. Each is recorded with its question. The Auto Ontology arms used revision `cdaf61e` with a twelve-file compatibility patch and took a median of about four minutes per question (three to five and a half by setting); arms A to S took one to three seconds and cost under a dollar in total.

An earlier experiment with 38 author-written questions about Auto Ontology alone, one run per arm, found the same direction for the table descriptions and shaped this design; its records are kept with the earlier protocol and results.

5. What the code database records

Before the earlier experiment, two extractor defects were fixed and its database rebuilt: imports now bind only in the scope that contains them, and calls in a nested function count for its enclosing function only when that function calls or passes on the nested one. The rebuild removed 21 of 9,442 linked calls and added 7; the question 11 answer did not change.

Every call site is classified, and a call the extractor cannot follow keeps no target rather than a guessed one. In that earlier snapshot of Auto Ontology, of 31,178 direct call sites, 9,442 resolve to an internal callee and 11,794 are classified as dynamic attribute calls without one. Another 1,046 module-level calls are omitted from the call table. The small residual category named `unresolved` does not measure all missing edges. Callback records have lower confidence; test links reflect imports or matching filenames rather than measured coverage. Repeated qualified definitions, such as property getters and setters, receive distinct line-qualified identities, while ambiguous targets remain unresolved.

Queryable source representations are established techniques. CodeQL builds databases for source analysis, and CodexGraph lets language-model agents query code graph databases. The contribution here is the adapter to Auto Ontology's contracts and its demonstrated use on source-derived rows.

6. Reviewing and extending the integration

For an Auto Ontology maintainer, the work separates into three reviewable pieces:

Review question	Implementation or evidence
How do source facts enter the existing interfaces?	Extractor, exporters and interface specification.
What did the live service need?	Compatibility patch, with configuration, boundaries and scoped tests.
What did the agent actually produce?	Detailed results, linking every run, response and score.

Software repositories already work as a semantic-catalogue source through Auto Ontology's existing connector and import. The open questions for its maintainers are whether keys and joins declared in an imported model are a supported contract for an outside adapter, and how public-model configuration and SQL routing should be supported.

The next tests are questions written by the repositories' maintainers, a comparison against an agent reading the source directly, and the semantics layer against a current Auto Ontology revision.

Separate governance commands record certification history, annotate SQL and reconcile definitions. They do not enforce certified-only retrieval or prove which definitions the agent used. This evaluation makes no new claim about those commands.

The study's Auto Ontology runs used Auto Ontology `cdaf61e`; every run's manifest in `study/runs/` names the bridge revision it started from, all at or after the commit that froze the study, `37b4f05`. The earlier comparison used bridge `2d3686c`, with its two DeepSeek Auto Ontology runs starting at `e1d41b3`, which changed only the website and the results summariser. Each Auto Ontology run's manifest binds full revisions, model settings, the database digest and frozen inputs. Follow the reproduction guide for extraction, compatibility tests or a new live run. Figure notes document the illustrations' sources and omissions.